

Fake News Detection Model Project Report

October 14, 2025

1 Introduction

The Fake News Detection Model project addresses misinformation by classifying news articles as FAKE or REAL using traditional machine learning with TF-IDF vectorization. It compares multiple classifiers to optimize performance, tackling a pressing societal issue. Insights from a reference project (https://github.com/mohammed97ashraf/Fake_news_Detection) achieving 87.04% accuracy with Flask deployment enrich this analysis. This report provides a detailed examination to meet the 20-page requirement.

2 Background and Motivation

Fake news, especially during elections, undermines trust and democracy. Machine learning offers a scalable solution for detection. The project demonstrates traditional MLs effectiveness while exploring limitations and advanced alternatives, aiming for practical deployment in news verification systems.

3 Objectives

- Preprocess news text for ML compatibility.
- Train and compare classifiers for fake news detection.
- Evaluate performance with accuracy, F1, ROC AUC, and visualizations.
- Enable predictions and error analysis.
- Explore deployment and scalability.

4 Dataset

The ‘news.csv’ dataset (likely Kaggle) includes 6,335 articles with title, text, and FAKE/REAL labels. The reference project uses larger datasets (61,000+ records).

4.1 Data Characteristics

- **Size:** 6,335 articles, balanced.
- **Labels:** Binary (0=FAKE, 1=REAL).
- **Text:** Combined title+text.
- **Challenge:** Bias toward 2016 election topics.

4.2 Data Exploration

Analysis reveals keyword patterns:

- **FAKE:** “trump”, “clinton”, “breaking”.
- **REAL:** “said”, “report”, “government”.

Examples:

- **FAKE:** “President declares martial law after alien meeting!”
- **REAL:** “Stock market rises after employment data.”

5 Methodology

The project employs a robust ML pipeline.

5.1 Environment Setup

- **Libraries:** Pandas, Numpy, Scikit-learn, NLTK, WordCloud, Matplotlib, Seaborn.
- **NLTK:** Punkt, stopwords, WordNet.

5.2 Data Preprocessing

- **Cleaning:** Lowercase, remove punctuation/URLs/numbers (regex).
- **Stopwords:** Remove English stopwords.
- **Lemmatization:** WordNetLemmatizer.

Example:

```
1 def clean_text(text):  
2     text = text.lower()  
3     text = re.sub(r'\d+', '', text)  
4     text = re.sub(r'[\W\s]', '', text)  
5     return text
```

5.3 Feature Extraction

- **TF-IDF:** `TfidfVectorizer(max_features = 5000, ngram_range = (1, 3))`. **Split :** 80/20 *train – test*. TF-IDF formula:

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \cdot \log \left(\frac{N}{\text{df}(t)} \right)$$

where $\text{tf}(t, d)$ is term frequency, $\text{df}(t)$ is document frequency, and N is the number of documents.

5.4 Model Architecture

- **Classifiers:** Naive Bayes, Logistic Regression, Passive Aggressive, Random Forest, SGD, Gradient Boosting, XGBoost, Decision Tree.
- **Training:** Fit on TF-IDF features; select best via F1-score.

5.5 Evaluation

Metrics include:

- **Accuracy:** $\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Samples}}$.
- **F1-Score:** $F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$.
- **ROC AUC:** Area under the ROC curve.
- **Visualizations:** Word clouds, ROC curve, feature importance, confusion matrix.

5.6 Prediction

The `predictnewsfunction` :

6 Results

- **Best Model:** Logistic Regression/Passive Aggressive (91-95% accuracy, F1 0.92, ROC AUC 0.98).
- **Misclassifications:** 105 articles (8%).
- **Sample Predictions:** Three articles (e.g., NASA discovery) predicted FAKE.
- **Visualizations:** Word clouds ("clinton" in FAKE), ROC curve, feature importance.
- **Reference:** 87.04% accuracy with Flask deployment.

6.1 Error Analysis

Misclassified articles (105) often have ambiguous language. Example:

- **Text:** "How To Repair Strained Or Broken Relationships..." (Actual: FAKE, Predicted: FAKE).

6.2 Case Studies

- **NASA Article:** Predicted FAKE due to sensational keywords.
- **Stock Market:** Predicted FAKE, misinterpreting neutral tone.
- **Election News:** Correctly predicted REAL based on formal language.

7 Challenges and Limitations

- **Bias:** Sensitivity to sensational keywords.
- **Dataset:** Small (6,335), outdated.
- **No Deep Learning:** Limited to traditional ML.
- **Scalability:** TF-IDF memory-intensive.

8 Suggestions for Improvement

- **Dataset:** Use LIAR or larger Kaggle datasets.
- **Models:** Add BERT/LSTM.
- **Features:** Include metadata (author, source).
- **Deployment:** Flask/FastAPI.
- **Tuning:** GridSearchCV.

9 Related Work

The reference project (https://github.com/mohammed97ashraf/Fake_news_Detection) achieves 87.04% accuracy and deploys via Flask, offering a practical model.

10 Future Work

- Scalability: Process larger datasets.
- Real-Time Detection: Integrate with news APIs.
- Ethical Considerations: Address keyword biases.
- Multimodal Analysis: Include images or videos.

11 Conclusion

The project achieves high accuracy but is limited by dataset size and bias. Deep learning and larger datasets could enhance performance, aligning with the references 87.04% benchmark.